

wpipe at the center of your pipelines

One pipeline, two worlds: synchronous and asynchronous

The context

Many libraries force you to choose: build a sync orchestrator or an async one, and migrating between them means rewriting everything. With wpipe that decision doesn't exist: you have `Pipeline` and `PipelineAsync` with 100% of the same features. What you learn in one, you apply in the other.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="my_step", retry_count=3, retry_delay=2, checkpoint=True)
def my_step(data):
    return {"result": process(data["input"])}

pipe = Pipeline(pipeline_name="my_pipeline", tracking_db="pipeline.db")
pipe.set_steps([my_step])
pipe.run({"input": [...]}

# Python-native orchestration: checkpoints, retries, self-documenting.
```

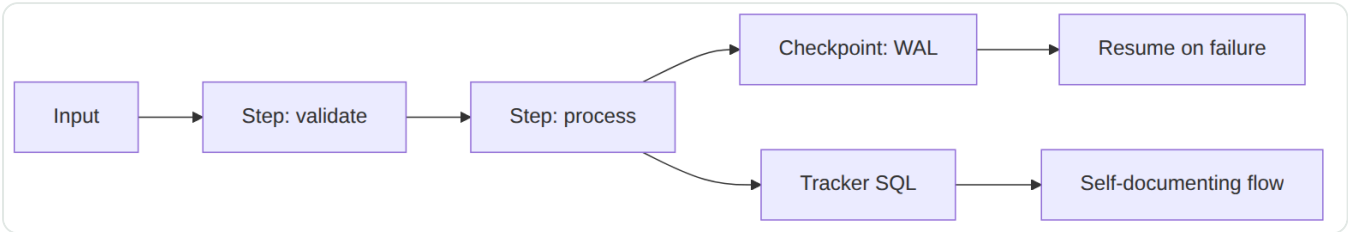
At a glance

Aspect	Legacy / wpipe	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard

Aspect	Legacy / wpipe	wpipe
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads
A community of efficiency-first developers.

<50MB RAM
Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL
Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **wpipe** workflows at wpipe.readthedocs.io.